# Minimalist coding for causal mapping


## Abstract

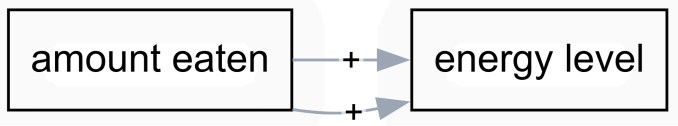
This working paper explains our **Minimalist / Barefoot** approach to coding causal claims in text as simple directed links ("X influenced Y"), developed through extensive experience with large-scale practical coding. We write it now because, although our previous work motivates causal mapping in evaluation (Powell et al. 2024), shows how QuIP-style "stories of change" elicit natively causal narrative evidence (Copestake et al. 2019), demonstrates ToC validation by comparing empirical maps with programme theory (Powell & Larquemin 2023), and shows that generative AI can extract links exhaustively with quotes as a low-level assistant (n.d.; Powell et al. 2025), none of these papers is a standalone, reader-facing account of **the coding stance itself**. Our approach is notable in particular because it rejects variable-based approaches used by most causal mapping traditions, which share many assumptions with rules used to build Systems Diagrams, Fuzzy Cognitive Maps, Causal Loop Diagrams, etc. We give an alternative, more primitive account of what exactly counts as a coded causal claim. We explain what we deliberately do *not* encode such as strength and polarity, and we mention the limits of this approach. **Intended audience:** evaluators, academics and qualitative researchers who want a citeable and teachable causal coding protocol, and AI/NLP readers who want an auditable way to identify and process causal content in text.


## Introduction: Why minimalist coding?

### *The variable-based approach*

1. I was eating less and felt quite lethargic
2. I started to eat adequately and was feeling more lively

To the best of our knowledge, all major approaches to causal mapping (Axelrod 1976; Eden et al. 1992; Laukkanen & Wang 2016; Maule et al. 2003) would most likely code (1) as something like `amount eaten` —> `energy level`. And they would treat (2) pretty much the same.




*Variable-based coding*

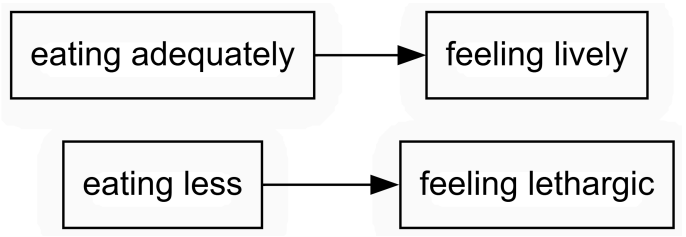
(Nomenclature: we will call all cause and/or effect labels "factor labels", whether in the variable-based or minimalist approaches.)

To make this work, we have to believe there are at least two things called variables which can take different values (higher/lower?) and that causal effects are powered by at some kind of (monotonic?) relationship between them: something like, *the more* I eat, *the more* energy I have (in addition to which we have to code, somewhere, four more actual, factual claims: I did eat X amount, and I did have Y energy levels).

Bizarrely, this also means that (1) and (2) are treated as essentially *the same*. They both affirm a "positive" influence of the first variable on the second: more of X means more of Y.

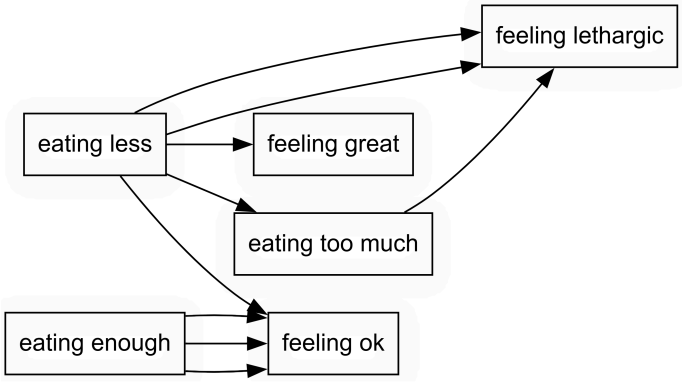
But is that what the speakers *meant*? How do we know if the speakers have say a continuous or Boolean (yes/no) model of "eating"? If Boolean, what is the opposite of eating a lot? Eating only a little? Eating less? Not eating? If continuous, how do we know what kind of function they use in their own internal model?

The variable-based model has to really batter reality into a brutally inadequate shape, and at the same time makes us use formal machinery which we maybe don't need or want.




*A propositional, minimalist coding of the same information*

Reading multiple texts on the same topic, or different diary entries from the same person, we will probably find much richer narratives which do not fit into those kinds of shapes at all. People eat too little, and then eat too much, and express different ideas of adequacy and normalcy at different times in different ways. Is "feeling lethargic" just low value on the same variable as "feeling great"? Who knows?




*An example of propositional, minimalist coding of more texts on the same theme*

In other words, to use the variable-based approach, we mostly have to go way beyond what the speaker actually meant; so it is not really a form of qualitative data analysis with the aim of modelling sources' beliefs. It's not just coding, it's modelling, with an ambiguity about whether we are trying to model the world or individuals' putative internal models of it.

We have found that it is often over-specified (and often psychologically implausible) to treat ordinary-language causal claims as if they asserted an explicit functional relationship between well-defined variables. Trying to force that kind of structure on everything turns the “easy 90%” of coding into a harder and more arbitrary task. Of course, one can decide to use a particular non-minimalist, perhaps variable-based representation for a particular modelling purpose; our claim is only that this is usually not a faithful representation of what most speakers actually say or imply, most of the time. For example, if we code "I got really tired because I have Long Covid", we could perhaps treat both endpoints as Boolean variables, but what about "I got really tired because it was really hot", and "I got really tired because it was really cold" — how are we going to represent “temperature” as a single variable while preserving the speaker’s intended meaning? If what we want to do is model a system, we can pick a solution. But if we want to model *cognition/beliefs/claims as expressed in text*, many “variable semantics” choices are over-committed.

Charles Ragin (2008) and others have done admirable thinking on Boolean values and opposites and have constructed sophisticated tools with somewhat simpler ontological baggage (set membership rather than values-of-variables) which perhaps model sentences like (1) and (2) somewhat better. But for many purposes we don't need that baggage either to code ordinary-language claims about causation. Let's see how far we can get with the minimal possible formal baggage.

Most causal mappers have always been quite conscious that modelling people's causal beliefs is not the same as modelling the real world. Sometimes the approach (Axelrod 1976) is explicitly designed to model someone's thinking. But often it is hard to keep the two worlds separate, what we have called the Janus problem (Powell et al. 2024, p. x). Modelling the real world is exciting, and useful. Traditions like Systems Diagrams, Fuzzy Cognitive Maps, Causal Loop Diagrams and Directed Acyclic Graphs all, to varying degrees, treat the diagram as a model of how the world works rather than a record of what people said. Variables, weights, and (sometimes) functional forms slip in, because the diagram itself is doing the modelling. Our claim is weaker. We are recording what people said about causes, with quotes, and leaving the question of how the world really works to a later step.

Using variable-based approaches allows one to code linear or even non-linear causal influences of single or even multiple causes on their effects. One can do the "coding" by simply writing down (using appropriate special syntax) the connections, because one is an expert, and/or one can verify such statements statistically on the basis of observational data. Thus armed, one can make predictions or have sophisticated arguments about counterfactuals.

## *The minimalist approach*

But using minimalist coding we cannot do that, because our claims are formally weaker and therefore our inference rules are weaker. What we *can* do is still really interesting. We can ask and answer many different kinds of useful questions like:

- what are the main influences on (or effects of) a particular factor, according to the sources?
- what are the upstream, indirect influences on (or effects of) a particular factor, bearing in mind The transitivity trap?
- how well is a given programme theory validated by the respondents' narratives? (We can do this basically by using embeddings to get measures of semantic similarity between labels and aggregate these as a goodness of fit of theory to data.)

That is all exciting and useful. It's a surprisingly simple way to make a lot of sense out of a lot of texts which is, with caveats, almost completely automatable.

**We start by describing the "minimalist" approach** to coding causal statements used for QuIP, developed originally by James, Fiona and colleagues at Bath SDR and further formalised at Causal Map Ltd in collaboration with Bath SDR. This formalisation lives inside the Causal Map app. We then set out the coding principles, position the approach within qualitative research, show how it can be extended with useful transformations, and finally ask how it copes with harder constructions like enabling, blocking and causal packages.

The minimalist approach is notable because it is based in our **joint experience of coding thousands and thousands of stakeholder interviews and other data such as project reports**, mostly from international development and related sectors, as well as coding hundreds of thousands of pages with AI-assisted coding. These have nearly always involved **multiple sources talking about at least partially overlapping subject matter**. So this coding produces individual causal maps for each source, which can then be combined in various ways.

### Contrast: multiple sources

This is in contrast to methods of constructing single-source maps of expert thinking (Axelrod 1976) or the collective construction of a consensus map ({Barbrook-Johnson 2022).

### Contrast: deferred judgement

Our approach is also different in emphasis from the kind of approach adopted by Jewlya Lynn (2025) and other evaluators (it is perhaps implied in Aston (2019) and in Contribution Analysis more broadly). That approach usually **first gathers all the evidence for any influence of X and Y prior to actual mapping**. So before a link from X to Y is actually formally coded it has already been subject to important evaluative judgement: are these testimonies, from these different sources in this context, enough for us to draw a link, and if so with what other attributes (e.g. this link may optionally also be judged to have strength 3 or relevance 2 or whatever).

Whereas our default approach is to code all causal claims willy-nilly: this often means that we code multiple, not necessarily compatible, claims between X and Y, creating sets of *co-terminal links* (sets with X at one end and Y at the other); we call such a set a *bundle*. Our maps show only a single arrow from X to Y but in the database there are actually multiple parallel claims. We defer judgement until later about the quality, compatibility and relevance of these claims and what they actually contribute to answering research or evaluation questions. We often report raw statistics e.g. about the number of claims before making any such judgement. See Quality assurance at each step of the causal coding workflow.

### Tasks

We treat causal mapping as three tasks (Powell et al. 2024).

- Task 1, data gathering, not covered here;
- Task 2, coding: creating **evidence-with-provenance** (a links table)
- Task 3, analysis: a sequence of transforms of the original links table: the final interpretation of the analysis can be

understood as a concatenation of the interpretation rules for each transform.

## Project context (how this paper fits)

A companion formalisation paper A formalisation of causal mapping makes key parts of the method more precise (data structures, constraints, and conservative inference rules), but the focus here is the narrative rationale and practical coding guidance.

**Unique contribution (what this paper adds):**

- A definition of the **minimalist/barefoot** coding stance ("X influenced Y, with provenance") and what it deliberately does *not* encode.
- An account of why this stance is useful at scale (including AI-assisted extraction) and where it fails (enablers/blockers; conjunctions/packages).
- Examples of additional extensions / transforms, called "filters" in the Causal Map app. Both the initial definition of a links table and each additional filter is described in terms of syntax and of semantics (interpretation rules).

This paper is part of a small set of new working papers

- Companion formal spec: A formalisation of causal mapping
- QDA positioning: Causal mapping as causal QDA
- Practical transforms/diagnostics: Magnetisation; A simple measure of the goodness of fit of a causal theory to a text corpus; Combining opposites, sentiment; other transforms to come later
- A worked "AI clerk vs human architect" example: Assessing change in (cognitive models of) systems over time

From the four prior papers already cited in the abstract, we adopt one stance in particular (Powell et al. 2024): causal mapping is primarily a way to assemble and organise **evidence-with-provenance**, keeping the subsequent evaluative judgement about "what is really happening" distinct. A coded link is first and foremost "there is evidence that a source claims X influenced Y", not a system model with weights or effect sizes. The present paper extracts and clarifies the core "minimalist" coding commitments that make that workflow, including its AI-assisted form (n.d.; Powell et al. 2025), workable and checkable.

Our experience has been that the vast majority of causal claims in these kinds of texts are easily and satisfactorily coded in the simplest possible form "X causally influenced Y". Explicit invocation of concepts like enabling/blocking, or necessary and/or sufficient conditions, or linear or even non-linear functions, or packages of causes, or even the strength of a link, are relatively rare. The causes and effects are not conceived of as variables, the causal link is undifferentiated, without even polarity, and if any counterfactual is implied it remains very unclear.

This approach is what we call **"Minimalist" or "Barefoot" Coding**.

# Minimalist coding principles

## The 90% rule

We have found that it is pretty easy to agree how to apply minimalist coding to say 90% of explicit causal claims in texts, without missing out essential causal information, whereas it is very difficult to find appropriate frameworks to cope with the remaining 10%.

## Fewest assumptions

Minimalist coding is perhaps the most primitive possible form of causal coding which makes no assumptions about the ontological form of the causal factors involved (the "causes" and "effects") or about *how* causes influence effects. In particular we do not have to decide if cause and/or effect is perhaps Boolean or ordinal, or if perhaps multiple causes belong in some kind of package or if there is some kind of specific functional relationship between causes and effects.

An act of causal coding is simply adding a link to a database or list of links: a link consists of **one new or reused cause label and one new or reused effect label**, together with the highlighted quote and the ID of the source.

Take the second diary entry from the start of this paper:

> I started to eat adequately and was feeling more lively

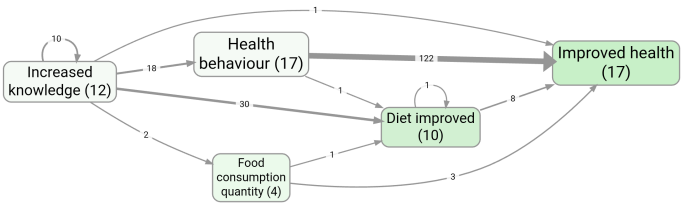
It can be trivially coded minimalist-style as

> I started to eat adequately —> feeling more lively (Source ID: diary entry 2; Quote: I started to eat adequately and was feeling more lively)

That's it.

## Causal maps

Crucially, we can then display the coded claims for individuals as a graphical causal map, and we can also display the entire map for all individuals and/or maps filtered in different ways to answer different questions. There is a handful of other applications (Ackermann et al. 1996; Laukkanen 2012) for causal mapping which also do this; but as far as we know, only Causal Map also allows direct QDA-style causal coding of texts.




*Bookmark #266 — a typical first-pass map: top five factors by frequency, with no further encoding of polarity, weight, or function. The numbers are evidence counts (citations on links, source counts on factors), not effect sizes.*

## Data structure

Although we have the option of creating additional tags associated with each link (where many approaches would for example code the polarity of a link) this is not central to our approach.

We don't use a separate native table for factor labels: they are simply derived on the fly from whatever labels happen to be used in the current table of links. This makes data processing simpler and also suggests an ontological stance: causal factors only exist in virtue of being part of causal claims.

We do however have an additional table for source metadata including the IDs of sources, which can be joined to the links table in order, for example, to be able to say "show me all the claims made by women".

## Causal powers

When a source uses causal language, we treat it as a claim that one thing made a difference to another (rather than a mere temporal sequence) *in virtue of its causal power to do so*.

## Causal influence, not determination

We believe that it's rare for people to make claims about causal determination: someone can say that eating properly again made them feel more lively, and then also agree that some good sleep had a lot to do with it too, without this feeling like a contradiction.

## Not even polarity

We differ even from most other approaches which are explicitly called "causal mapping" in that we do not even "out of the box" record polarity of links (to do so would involve making assumptions about the nature of the "variables" at each end of the link as well the function from one to the other).

## The Focus on Cognition

In the minimalist approach, **we are quite clear that what we are trying to code is the speaker's surface cognitions and causal thinking**, while the actual reality of the things themselves is simply bracketed off at this stage, either to be revisited later (because we are indeed interested in the facts beyond the claims) or not (because we are anyway interested in the cognitions).

## Staying on the surface

At Causal Map, we rarely make any effort to get beneath the surface, to try to infer hidden or implicit meanings. This is particularly well-suited to coding at scale and/or with AI. Our colleagues at Bath SDR do this a bit differently, spending more effort to read across an entire source to work out what the source *really* meant to say.

## Closer to the cognitive truth

People rarely talk about causes as variables. In our experience they say "X made Y happen", or "Y was because of X", and that is most of what they say. They almost never commit to whether X is continuous or Boolean, or to whether the relationship is linear, threshold, or anything else. If we are trying to record what they said, we are better off not adding any machinery they did not specify. That is what we mean by staying close to the cognitive surface: the labels carry whatever the speaker was carrying, and we add nothing else.

## Unclear counterfactuals

A causal claim in ordinary speech rarely carries a clean counterfactual. "I started to eat adequately and was feeling more lively" doesn't tell us whether the speaker thinks they would have felt only slightly more lively without eating properly, or would have stayed lethargic, or whether the comparison wouldn't apply because their circumstances would have been different anyway. Variable-based coding has to commit to one of these answers, even if only by default. Minimalist coding doesn't. A coded link says "X influenced Y", and the counterfactual stays where the speaker left it.

## General versus specific

Minimalist coding focuses primarily on **factual causal claims** which also warrant the inference that both X and Y actually happened / were the case.

Most causal claims in the kinds of texts we have dealt with (interviews and published or internal reports in international development and some other sectors) are factual, about the present or past. Sometimes we see general claims, and we often just code these willy-nilly. In any case, the distinction between general claims and claims about specific events that actually happen is often fractal (a general claim can seem specific in a broader context) and difficult to maintain completely when modelling ordinary language.

## We don't code absences

Minimalist coding may be reasonably also called **Qualitative Causal Coding** or **Causal QDA coding**. It shares characteristics with some forms of coding within Qualitative Data Analysis (QDA), in particular demonstrating an asymmetry between presence and absence (a specific tag not being applied is not the same as the application of an "oppositely-poled" tag).

We do not code absences unless they are specified within the text (e.g. perhaps "because of the barking dog, the owner did not come out of the house").

While codes may be counted, the concept of a *proportion* of codes is challenging because the denominator is often unclear.

If families are talking about reasons for family disputes, and family F mentions social media use, and family G mentions homework, we do not usually interpret this as meaning that family F does *not* think that homework can also be a cause of family disputes. (This should derive formally from the interpretation of multiple causal claims).

## The labels do all the work

At Causal Map Ltd, our canonical methodology initially involves in vivo coding, using the actual words in the text as factor labels. This initial process generates hundreds of overlapping factor labels. This part is really easy (and is easy to automate with AI). Obviously, hundreds (or hundreds of thousands) of overlapping factor labels are not very useful, so we need to somehow consolidate them. Arguably, minimalist coding makes the initial coding easy but it just defers some of the challenges to the recoding phase.

One way to code additional meaning within factor labels is to add **tags as literal text inside the label**.

Tags can be used to group factors into **themes** which are not themselves causal factors (e.g. health, environment), without making them part of a hierarchy. They can add **lightweight metadata** for later filtering (e.g. `(Outcome)`, `#nutrition`, `[Work]`), while keeping the underlying causal claim as just `X -> Y`.

Suitable filters can then be provided to:

- **Filter by theme/flag**: because tags are just text, any "match start/anywhere/exact" label filter can include or exclude factors containing `#health`, `(Outcome)`, etc.
- **Hide tags for display**: to strip tags out for a cleaner map/table view, while retaining them in the underlying data.

Tagging factor labels is different from adding link metadata because it literally creates different causal factors. So `Improved health` and `Improved health [outcome]` are formally two different causal factors. However if we apply a filter to strip the `[outcome]` tag, they then become the same causal factor.

For a fuller discussion (including examples like `#nutrition` and `(Outcome)` and why uniqueness matters), see: Factor label tags — coding factor metadata within its label.

### *Coping with many labels*

We can:

- Use human- or AI-powered clustering techniques to consolidate the codes according to some theory or iteratively according to some developing theory
- Use AI-powered clustering techniques to consolidate the codes according to automated, numerical encoding of their meanings
- "Hard-recode" the entire dataset using a newly agreed codebook (see above)
- "Soft-recode" the dataset on the fly using embeddings to recode raw labels into those codebook labels to which they are most similar

## Evidence strength is not causal effect size

Counts (how many times a link is coded; how many sources mention it) measure *evidence volume/breadth in the corpus*, not causal magnitude in the world. This matters because the outputs can look like a quantitative system model even when they are not one.

In the app we routinely distinguish:

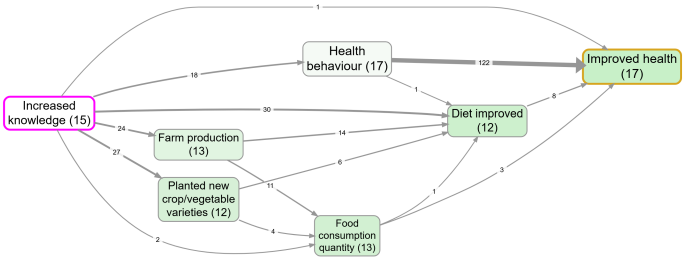
- **Citation count**: how many coded mentions support a link (volume).
- **Source count**: how many distinct sources mention it (breadth / rough consensus).

These are useful for prioritising what to look at, or for filtering (e.g. keeping only links with `min_source_count = 2`), but they do not tell you whether a causal influence is "stronger" in any effect-size sense.

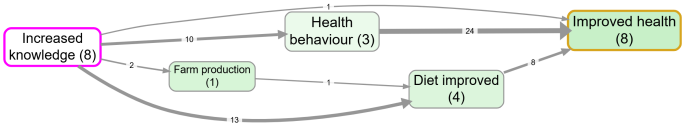
## The transitivity trap and "thread tracing"

It is usually invalid to infer a long causal chain by stitching together links from different sources. A conservative rule is: only treat an indirect pathway $A \to B \to C$ as supported when the *same source* provides each step ("thread tracing"), unless contexts are explicitly aligned.

The canonical failure mode is: source 1 says `A -> B`, source 2 says `B -> C`, and we mistakenly conclude that anyone told a coherent story `A -> B -> C`. In practice, for a given query we loop through sources one at a time, construct the valid paths *within each source*, and then combine only the edges that appear in valid within-source paths.




*Bookmark #1129 — path tracing from `Increased knowledge` to `Improved health`: every link on a route between them, across all sources. Easy to misread as a story anyone told.*




*Bookmark #981 — same trace with source tracing on. Only routes told within a single source survive: a much smaller, more defensible set of edges.*

## The filter pipeline as a mental model for analysis

Most analyses are built by applying a sequence of simple operations (for example: subset links to retain only specific sources; trace paths; rewrite labels; bundle duplicates; then show a map/table). Thinking in terms of a pipeline helps make the meaning of an output explicit: it is always “the result of these filters, in this order”.

Concretely, you can think of an analysis as “a links table passed through transforms”, where `|>` is a "pipe" symbol which just passes a result from the left to a new transform on the right.

e.g.:

```
Links |> filter_sources(...) |> trace_paths(...) |>
transform_labels(zoom) |> combine_opposites |> bundle_links |>
map_view
```

This is also why “the same dataset” can yield very different-looking maps without any contradiction: you are simply looking at different derived views defined by different pipelines.

So one can think of analysis as a sequence of transforms of the original links table. The final interpretation of the analysis can be understood as a concatenation of the interpretation rules for each transform.

This pipeline also exists outside the app as a standalone open-source package, (`n.d., /filter-engine`) (https://www.npmjs.com/package/(n.d.)/filter-engine), which takes a links table in, applies an ordered sequence of these filters (label and tag filters, path tracing, frequency thresholds, combine-opposites, zoom and more), and returns a links table out.

## Positioning within qualitative research (and likely critiques)

This paper is about a **coding stance** and a corresponding **intermediate representation** (a links table with provenance), not a claim that “coding = analysis”. In standard QDA terms, minimalist causal coding is best understood as a disciplined way to build an auditable evidence base that can later be interpreted, queried, and written up (Miles et al. 2014; Salda{\~n 2015). It is, of course, not the only kind of way to do QDA.

To reduce avoidable points of attack from mainstream qualitative social science readers, we make four explicit commitments up front:

1. **We code claims, not causal truth.** A coded link is evidence that a source *claimed* an influence relation. It is not (by itself) a causal inference claim about the world. This keeps the method compatible with both realist and constructivist sensibilities (n.d.; Charmaz 2014).
2. **We do not treat “counts” as effect sizes.** Counting supports prioritisation and transparency, but it is not a substitute for interpretation; frequency/breadth in a corpus is not magnitude in the world. (See "Evidence strength is not causal effect size" above.)
3. **We trade interpretive depth for auditability and scale, on purpose.** We stay close to surface causal language and preserve provenance (quotes + source ids) so that readers can check what is being claimed. This is a pragmatic stance when working with many sources and/or AI assistance; it does not deny that richer interpretive work can be valuable.
4. **Minimalist coding is not a full QDA workflow.** It is one layer that can sit alongside thematic analysis or qualitative content analysis when those are needed (Braun & Clarke 2021; Mayring 2000).

### *How a qualitative-methods reviewer might criticise this (and our response)*

Below are common criticisms (often reasonable) and the corresponding guardrails/defences built into the minimalist stance.

**Critique 1: “This is reductionist / strips context / decontextualises quotes”**

**Response:** We keep provenance as first-class data (source id + quote) and treat every map/table view as a derived view of that evidence. Context is handled explicitly by joining source metadata (e.g. subgroup, time) and filtering the links table; it is not “assumed away”. When deeper within-case interpretation matters, the same links table can be re-read within full-text context, and the write-up remains responsible for integrating that context (Miles et al. 2014).

**Critique 2: “You are smuggling in a positivist epistemology (or a realist metaphysics)”**

**Response:** The method’s core object is **reported causal thinking** (surface causal claims), not an ontic causal model. We therefore keep the epistemic claim modest: “this is what sources said, with quotes”, and we separate that from subsequent judgement about what is happening. That separation is consistent with standard qualitative criteria emphasising transparency, audit trails, and reflexive interpretation (n.d.; Charmaz 2014).

**Critique 3: “Coding is not analysis; links don’t ‘explain’ anything”**

**Response:** Agreed. Minimalist causal coding produces a structured evidence base that supports multiple analyses (filters, comparisons, pathway queries) and supports more disciplined writing, but it does not replace interpretation. In practice, it can reduce time spent on low-level organisation of text so that more time can go into interpretation and argument (Salda{\~n 2015; Gl{\"a 2013).

**Critique 4: “You are privileging causal language and missing other kinds of meaning”**

**Response:** Yes: by design. Minimalist coding is for projects where the core questions are themselves causal (mechanisms, pathways, theories of change). When non-causal meaning is central (identity work, norms, metaphors), other QDA approaches could lead; minimalist coding can still be a useful *additional* layer rather than the whole analysis ((Braun & Clarke 2021)).

**Critique 5: "Automatable coding invites false precision and overclaiming"**

**Response:** We explicitly constrain what automation is allowed to do: extract candidate links with quotes; keep everything auditable; and avoid treating the resulting network as a system model with polarity/weights "by default". The transparency of provenance is the main defence against black-box "AI did the analysis" claims.

# Extensions

There are many extensions one can add on top of minimalist coding. Small extensions can just add convenient functionality; other extensions can upgrade the system to other more fully-fledged kinds of coding like Fuzzy Cognitive Mapping.

An extension in general consists of syntax rules for how to construct and carry out a transformation, and semantic or interpretation rules for what this transformation means.

This paper treats in full only one extension: **hierarchical coding**, because it is simple, widely useful in practice, and it directly supports a transparent family of "zoom" transforms. The other extensions appear only as concise, standalone stubs; fuller treatments are published as separate work-in-progress papers.

## Some useful extensions

### *Hierarchical coding and "zooming" (FIL-ZOOM)*

#### Motivation

In any qualitative coding process there is a tension between **detail** (keeping what respondents actually said) and **summarisation** (making patterns visible). Hierarchical factor labels give us a simple way to keep both:

- We can keep a detailed factor such as `Healthy behaviour; hand washing`.
- We can also treat it as an instance of a higher-level causal factor `Healthy behaviour` when we want a higher-level view.

This is particularly useful in causal mapping because it lets us simplify a complex map *without changing the underlying link evidence*: we are simply rewriting labels into more general ones.

#### Coding conventions

We use the separator `;` to create nested factor labels, using a template like:

```
General concept; specific concept
```

For example:

```
New intervention; midwife training -> Healthy behaviour; hand washing
```

This is an example of what we called earlier "letting the labels do the work". As we do not in any case have a native factors table where we might record actual relationships between say lower-level and higher-level factors, we can do the same thing implicitly simply with the text of the label.

In AI-assisted coding, a practical way to encourage hierarchical labels is to explicitly instruct the coder (human or AI) to label each factor using the template **"general concept; specific concept"**, and to always provide a verbatim quote for each coded claim so every link remains checkable.

For example, an AI might produce a single coded link such as:

- `Improved agricultural practices; diversified crops -> Increased income generation; more sales`
  - Quote: "with a lot of good product, we are now able to sell more."

You can read the separator as:

- "A, and in particular B" (forward), or
- "B, which is an example of / part of A" (reverse).

This can be extended to multiple levels:

```
Improved hospital skills training; new midwife training; hand washing instructions improved
```

Two practical conveniences follow immediately:

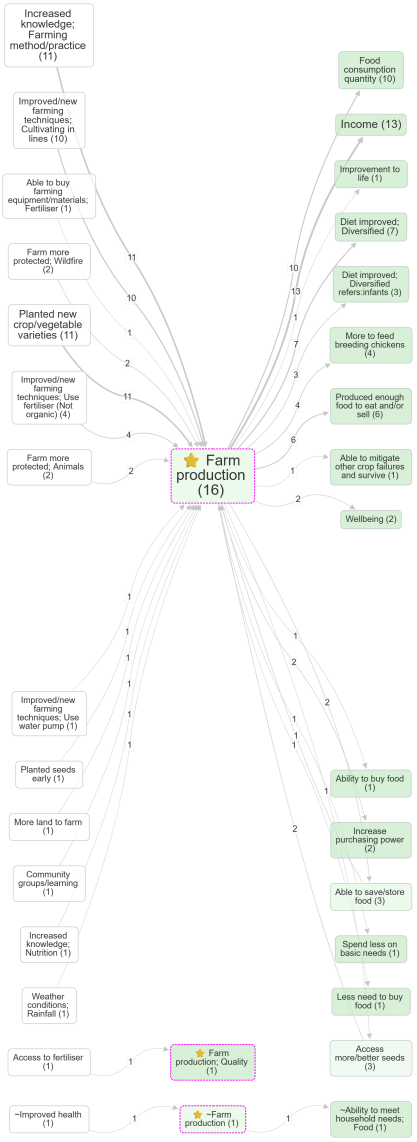
- Searching for the higher-level label (e.g. "Healthy behaviour") will also find its nested sublabels.
- Higher-level labels can be created and changed on the fly (they are just the visible prefixes before `;`), without maintaining a separate "parent code" structure.

#### Zooming and visualisation

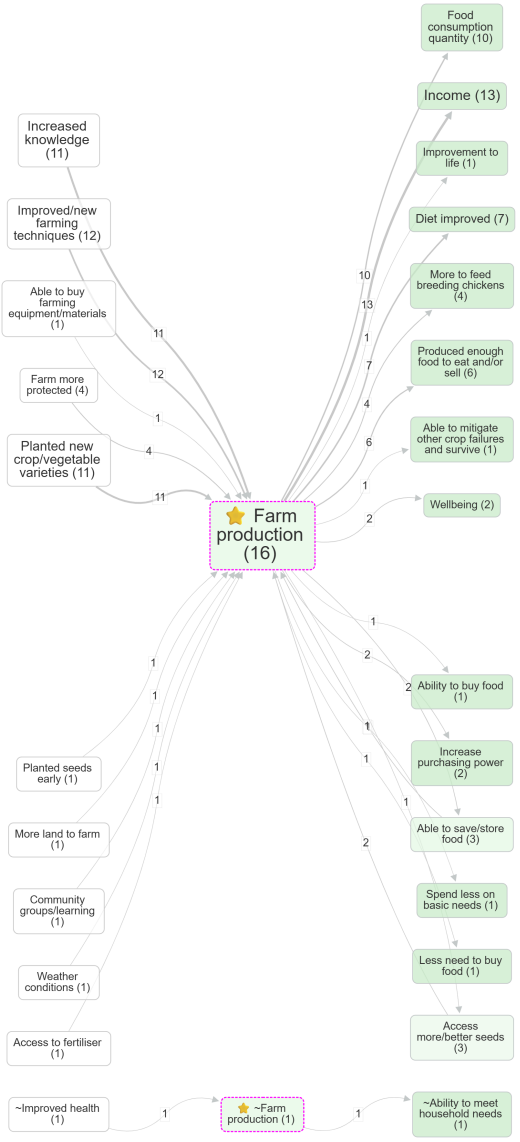
The basic idea of a zoom view is: **rewrite nested labels to a chosen level of abstraction**, then draw the map using the rewritten labels.

At "zoom level 1" we simply truncate at the first `;`:

- `Healthy behaviour; hand washing` becomes `Healthy behaviour`.




*Bookmark #805 — Focus filter on `farm production` with no zooming: detailed labels remain.*




*Bookmark #806 — same focus, zoomed to level 1: nested labels collapse to their top-level parents, the map gets more readable, and no underlying coding has changed.*

So if we have (at the detailed level):

`Improved hospital skills training; improved midwife training; hand washing instructions improved`

then we can show higher-level summary links such as:

- `Improved hospital skills training; improved midwife training`
- and (at the coarsest level) `Improved hospital skills training`

### Caveats (don't use the `;` separator mechanically)

Intuitively: by writing `A; B` you are explicitly licensing the interpretation "for high-level summary purposes, treat this as A".

You can also see this as a family of conservative "deductions" licensed by the `;` separator. We are explicitly allowing ourselves (for summary purposes) to treat the more detailed version also as evidence for the two less detailed versions: `Improved hospital skills training; improved midwife training; hand washing instructions improved` is also evidence for `Improved hospital skills training`.

The `;` separator encodes a commitment: that the more specific factor can be “rolled up” into the more general one without essentially distorting the causal story.

### Other tips

Try not to mix desirability within one hierarchy (e.g. avoid `Stakeholder capacity; lack of skills`, because zooming out would treat it as evidence for “Stakeholder capacity”). Use opposites coding (`~...`) for this kind of case.

A factor cannot belong to **two different** hierarchies at once (because there is no single parent to roll up into).

### When *not* to use a hierarchy (use a tag instead)

Don’t use `;` to express a non-causal *topic theme*. `A; B` explicitly licenses: “for high-level summary purposes, treat this as `A`”, so `A` must be a real (semi-quantitative) causal factor, not just a theme label like “Health”.

If something is merely health-related (with no natural roll-up into a single causal factor), use a tag instead:

- `Vaccinations law is passed [Health]`
- `Mortality rate [Health]`

Only use a hierarchy when the right-to-left label is genuinely a refinement of the left-to-right one (so roll-up makes sense), e.g.:

- `Improved health training; Improved hospital skills training`

## *Other extensions (stubs; treated in subsequent articles)*

The formal treatment of these is in A formalisation of causal mapping. Each stub below points to its own work-in-progress paper where one exists.

### Analysis filters (FIL-CTX, FIL-FREQUENCY, FIL-BUNDLE)

Three extensions are ordinary analysis filters in the pipeline sense above, and need no new coding conventions: restricting the evidence base by source metadata, for example only women or only a time period (FIL-CTX); retaining only links meeting an evidence threshold such as `min_source_count=2` (FIL-FREQUENCY); and computing evidence-strength columns such as `Citation_Count` and `Source_Count` for each `Cause -> Effect` pair, for use in map and table views (FIL-BUNDLE).

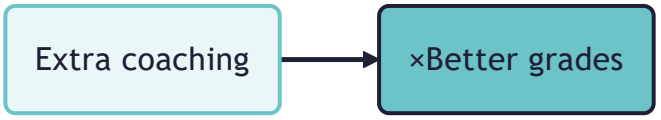
### Opposites coding (FIL-OPP) (stub)

Rather than encoding signed/weighted edges, we can get similar benefits by treating explicit opposites in labels (e.g. `~Employment` vs `Employment`) as a label-level device with simple inference and visualisation rules.

See Combining opposites, sentiment

### In-vain links (stub)

Narratives often name an influence that was aimed at an outcome which did not happen: "despite the extra coaching, her grades did not improve"; "the river still rose, despite the flood-prevention work". Coding an ordinary link asserts something the speaker denies; coding nothing throws the information away. So we code the outcome the influence was aimed at, with a cross at the very start of the effect label: `Extra coaching --> ×Better grades`, `Flood-prevention work --> ×~River levels rose`. Read the cross as "in vain". `×Better grades` is a separate factor from `Better grades`, so an in-vain link is never counted as evidence that the outcome happened, and it can be listed by searching labels for `×`.

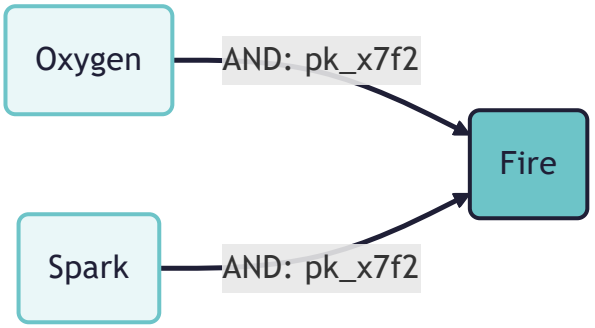



See In-vain links

### Coding packages (FIL-PACKAGE) (stub)

Sometimes a source presents several influences as working *together*: "you need both oxygen and a spark to make fire". Two separate links drop the conjunction; one combined label (`oxygen and a spark`) is no longer parsable. The package extension keeps the individual links and ties them together with two extra pieces of link metadata: a shared arbitrary `package` code, unique to that combination, and a `package_type` naming the connective, usually `AND` or `OR`, with `?` where the speaker grouped the causes in one breath without saying which. (`OR` is looser: alternative sufficient causes arguably need no package at all, so an `OR` package records only that the speaker offered the influences together as alternatives rather than asserting each outright; `?` is the honest default when even that much is unclear.) A package can also group the effects of one cause ("the programme raised awareness and changed behaviour"): this matters less for analysis, since effect links read separately rarely overstate the claim, but it is often the more natural reading, and it keeps the convention symmetrical: a package groups the links of one joint claim, whichever end the conjunction is on.

The semantics are weak by design: a package records that the speaker presented these influences as working together in the named manner, with no commitment to truth tables, necessity or sufficiency, interaction effects, or completeness. Packaged links remain ordinary links for every other purpose; the code makes the combination filterable (show all packaged links, one package, or one type) and countable without inflating conjunctive claims into independent evidence.

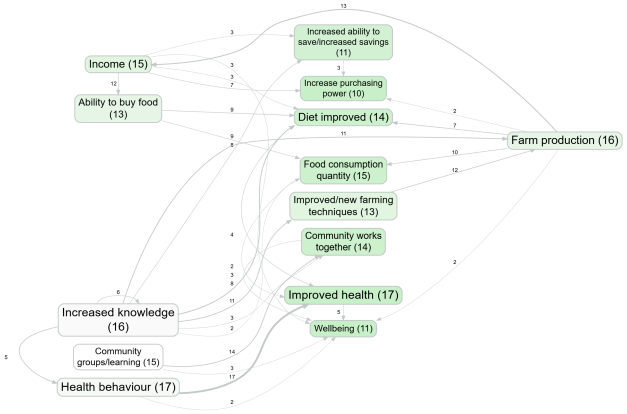



See Coding packages of links

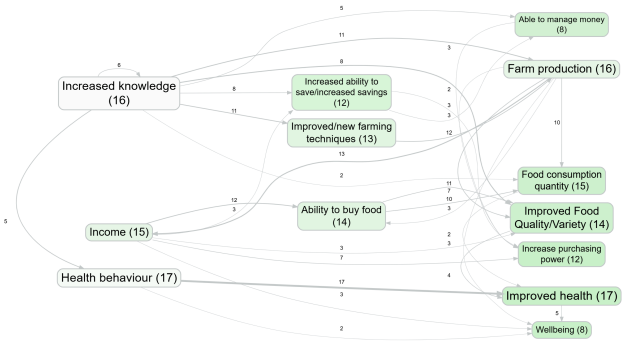
## AI extensions (optional; stubs)

### *Soft recoding (magnets) (FIL-SOFT)*

Many raw in-vivo labels can be aggregated by mapping them to a smaller vocabulary of “magnets” using semantic similarity. This changes labels (a recoding step), not the underlying minimalist meaning of a single coded link.




*Bookmark #493 — soft recode using all top-level labels from a human-coded version of the same project as magnets.*




*Bookmark #494 — same setup but recycling weaker matches. The change in settings shifts which raw labels attach to which magnets.*

### *Auto recoding (clustering) (FIL-AUTO)*

Alternatively, labels can be clustered into emergent groups and then rewritten accordingly, again as a recoding/aggregation step rather than a new causal logic.

## How other schools of causal mapping extend minimalist links

The “minimalist” stance in this paper is deliberately radical: we code *undifferentiated* causal influence claims with provenance, and then do most interpretation work via filters and views. Many other causal mapping traditions extend the representation in the opposite direction: they add more **semantic structure to factors and links**, especially **polarity** (and sometimes strength/weights), often with an implicit assumption that factors behave like **variables** and that a link expresses some kind of monotonic relationship. For a broader overview of these traditions and their differences, see (Powell et al. 2024).

### *What these extensions usually add*

- **Polarity (+/−)**: a link can mean “more of X leads to more of Y” (positive) or “more of X leads to less of Y” (negative).
- **Strength / weights**: a link can be assigned a magnitude (sometimes elicited, sometimes assigned by analysts), which invites quantitative reasoning and simulation-like interpretations.
- **Variable semantics and counterfactual clarity**: polarity and weights usually presuppose that the endpoints are comparable as variables (or at least as ordered states), so that “more/less” (or “increase/decrease”) is meaningful.

These extensions can be extremely useful in settings where respondents are explicitly reasoning in those terms, or where the purpose is modelling/simulation. But they also raise the bar: if we write down a signed or weighted link, we are committing not just to “X influenced Y”, but to a stronger claim about *how* X and Y vary and how changes propagate.

### *Three traditions, briefly*

In the Axelrod tradition, cognitive maps represent beliefs about causal influence among concepts and are usually coded with **positive/negative influence** as well as direction (Axelrod 1976; Axelrod 1976); this makes reinforcing and balancing structure visible, but moves the representation closer to variables-with-values even where the original elicitation was not precise about scales or functional form. In the Eden/Ackermann “cause maps” tradition, maps are built interactively to structure messy problems and support decisions (Eden et al. 1992; Ackermann et al. 2004; Ackermann et al. 1996); polarity is more natural there because the map is negotiated in a workshop, where the group can settle what “increase/decrease” means. Comparative causal mapping elicits, standardises and compares many maps across people, groups or time, which again pushes towards a more “variable-like” reading of factors so that maps can be aligned at scale (Laukkanen 1994; Laukkanen 2012; Markiczy & Goldberg 1995; Hodgkinson et al. 2004).

### *What it would take to extend our logic to signed links (and why we treat it separately)*

We *can* extend our links-table logic to incorporate polarity, but doing so is not just “adding a column”; it changes the semantics.

At minimum, we would need:

- A `Polarity` field per coded link (e.g. `+`, `-`, `unknown`) that is grounded in the source text or elicitation.
- A rule for how polarity is inferred when it is implicit, ambiguous, or mixed (which is common in narrative material).
- A clear semantics for what `+` and `-` *mean* (typically: a monotonic relationship between variables), including what counts as “more/less” for a factor label.
- Aggregation rules for how to deal with disagreement and contradiction across sources, and for how to visualise those

disagreements without creating spurious precision.

Because our target corpora typically do *not* make those commitments explicit (and because we are focused on modelling evidence-with-provenance rather than system dynamics), we do not treat signed links as part of the core method here. Where polarity matters in practice, we prefer to handle it with **label-level devices** (e.g. `~Employment` vs `Employment` combined with the `combine_opposites` transformation).

# Context and mechanism

Evaluators reading this will recognise two questions that any causal method has to answer. How do you code context? And what do you do about mechanisms? Both come up in realist evaluation and its Context-Mechanism-Outcome formula (Pawson & Tilley 1997). Minimalist coding answers both the same way. It gives them no special machinery and treats them as ordinary parts of the causal map. This is primitive and it flattens distinctions which can be important, but it is *relatively* easy to apply, and the results are not obviously worse than any more sophisticated approach.

## Coding context

Context is the standard worry. The same intervention has different effects in different places, and a good evaluation should say so. The realist instinct is to treat context as a separate kind of thing, a backdrop or a set of enabling conditions that decides whether a mechanism fires, distinct from the causes themselves.

The minimalist move is to code context as one more causal factor. If a source says the training worked because the community already trusted the facilitators, then `community already trusted the facilitators` is a factor, and it influenced the outcome, coded as an ordinary link. Nothing new is needed.

This is primitive. It drops the special status that context is often given, and it does not by itself mark the difference between a cause that triggers an effect and a condition that merely lets some other cause work. Where that difference really matters, it is the same second-order enabler problem we discuss below, and we handle it the same way, usually in the write-up.

Coding context as a factor is also not the only option. You can record it as link metadata (a column in the table of links, or in particular as a dedicated link tag like #afterCovid or #onlyInSevereCases) or as a factor tag (`Trust [afterCovid]`). We use each of these where it fits.

The idea of 'Context' also overlaps with the idea of 'Source'. *True in this context* is not so far from *True for this source*, and in our minimalist approach we have some surprisingly sophisticated ideas about dealing with what *True for this source* means when tracing causal influences upstream and downstream, see here. In other words, to the extent that *Source* is for you a reasonable proxy for *Context*, then you can use the fact that all your source metadata is already available to you in your links dataset, for example you can add a filter to show only causal stories which are *true for these particular sources*.

There is no universally accepted way to encode context because there is no even half-way accepted operational agreement about how to distinguish triggers, contexts and mechanisms. Realist evaluation has argued for decades about what counts as context as opposed to mechanism or outcome. Yes, these concepts can be usefully applied in a given case or study. But there are different ways to apply them. That is a strength as well as a problem. But the problem remains that you can't just give a set of instructions on how to encode them and be sure that they will be applied in the same way by different assistant coders (human or AI), Given that, coding context as a plain factor is a reasonable default. It is simple, and it defers to the analysis phase the harder judgement about what role context is playing.

## Coding mechanisms

The CMO formula raises a sharper question that the realist literature has never really settled, even after a dedicated review of how realist evaluations use the term (Lemire et al. 2020): what is a mechanism? Definitions range over hidden generative processes, the reasoning and resources of actors, underlying social structures, and several other things, and a coder who has to decide which one applies before coding anything will not get far.

Minimalist coding avoids the definition. A mechanism is just a piece of the causal map. If context factors E and F lead to outcomes X and Y, possibly through intermediate factors M and N, then the mechanism is that part of the graph: the links from E and F through M and N to X and Y. There is nothing extra to encode and no separate object called a mechanism. You read it off the map by tracing paths, with the same caution about combining links across sources discussed under source tracing.

This too is primitive and flattening. It does not capture the generative, what-is-really-going-on-underneath sense that realists want from the word mechanism, and it will not satisfy a reader who thinks a mechanism must be more than the connections people described. But it works. It gives you a concrete, checkable object made of coded claims with quotes, where the realist account gives you a long-running argument about definitions. If a richer notion of mechanism is needed for a particular study, it can be built on top, the same way polarity or packages can. The minimalist record does not block that, and it is a usable starting point in the meantime.

### *Causal packages / conjunctions*

This overlaps with the problem of how to code causal packages: claims in which some *combination* of factors is said to be needed for an effect. It's part of realist thinking, but it appears in other approaches too.

Our answer is the weak-by-design package extension (FIL-PACKAGE, above): the member links stay ordinary links, a shared code records the claimed combination, and the stronger model-like commitments (truth tables, interaction effects, necessity and sufficiency) are exactly what the extension refuses to make. See Coding packages of links.

## Where minimalist coding runs out

Minimalist coding handles maybe 90% of explicit causal claims in the kinds of texts we work with. As discussed above, it may not satisfy researchers who have a specific meta-theory they want to apply, like CMOs, or INUS packages.

The remaining 10% includes a few recognisable types: claims that are really about other claims (enablers and blockers), claims that depend on a combination of factors (causal packages, treated above), and claims that pack a sophisticated causal story into a single phrase ("went viral").

We've thought about all three. Beyond the two weak-by-design markers already described (in-vain links and package codes), none of them has led us to add new structure to the coding scheme. Here is why.

### Causation about causation: enablers and blockers as second-order causal claims

One natural way to try to go beyond truly minimalist coding is to say that some claims are not about simple factors causing other simple factors, but about one factor influencing (enabling or blocking) a *causal connection* between two other factors.

Consider an enabler-style statement:

> I went on holiday to Spain expecting to enjoy it, and indeed I did particularly enjoy it because I remembered to take my phrase book.

In strictly minimalist coding, we might record two undifferentiated links:

- Going on holiday to Spain —> Enjoying the holiday
- Remembering to take my phrase book —> Enjoying the holiday

But arguably the second claim is not really “the phrase book caused enjoyment” in the same way as the first. Rather, it enabled the normal causal power of the holiday to produce enjoyment. A more explicit encoding would therefore be:

1. Going on holiday to Spain —> Enjoying the holiday
2. Remembering to take my phrase book —> (Going on holiday to Spain —> Enjoying the holiday)

Visually, (2) would be drawn as an arrow pointing to the middle of the arrow in (1). Semantically, it can be read as:

> The phrase book enabled the causal link from X to Y, in virtue of its causal power to do so.

Blockers are closely related but show the asymmetry more starkly:

> I went on holiday to Spain, but I forgot to take my phrase book so I didn't enjoy the holiday at all.

One can code:

- Forgetting to take my phrase book —> Not enjoying the holiday

However, our intuition that “going on holiday” should also be part of the story is hard to capture without moving to a second-order encoding. The blocker case is visually similar to the enabler case, but it seems to require adding the idea of a causal power to *stop* something: the blocker destroys or disables the causal power of X to bring about Y. In practice we handle the related blocked-outcome case with in-vain links (above), which record a countervailing influence that was present and failed, without committing to a signed edge.

We do not claim we can do much with this at scale; but it is a clear way of stating what seems to be “missing” from purely first-order, undifferentiated links in these edge cases.

## Sophisticated claims as ordinary language

A more cheerful note. Some sophisticated causal constructions become part of ordinary language as a single phrase. "Her post mocking Farage went viral, so Farage was forced to respond" packs a fairly elaborate causal story into the verb "went viral", and a minimalist coder can take it as one factor without trying to unpack what "viral" really means. The same trick lets us code "she got cancelled", "the policy backfired", or "the deal fell through" as ordinary minimalist factors, deferring the unpacking to wherever it actually matters. The benefit is real: we don't need a new coding apparatus every time the world invents a new causal idiom. We just need labels that hold their meaning long enough to be useful. Or: reality is fractal.

## What this section adds up to

We could in principle extend minimalist coding to handle enablers, blockers, and packages with rich new structure. We have mostly not. The cost of a richer scheme almost never pays back, because the relevant cases are rare and the analyst can usually handle them adequately in the write-up, where they belong. We would rather log a slightly imperfect minimalist record and write nuance into the prose than build it into the data structure. The two exceptions we allow ourselves are weak-by-design markers on otherwise ordinary links: the in-vain mark on an effect and package codes (FIL-PACKAGE), each described above, and each refusing the stronger commitments that make richer schemes expensive.

See also our related work: the evaluator-facing overview (Powell et al. 2024), the AI-assisted coding papers (n.d.; Powell et al. 2025), the application to outcome harvesting (Britt et al. 2025), and the handbook chapter on qualitative causal mapping in evaluations (Remnant et al. 2025).

# Conclusion

Minimalist coding is proud of being unsophisticated. We code "X influenced Y, with provenance", and we let the labels do most of the work. We don't code polarity by default, we don't fit functions, and we don't worry about counterfactuals the speaker didn't supply. We give up the predictive and simulation-style affordances of more elaborate coding traditions in exchange for something more practical: an auditable evidence base that scales to thousands of sources, works well and quickly with AI assistance, and stays close to what people actually said.

We've used this stance across consultancy and research work for years now, and it keeps repaying its simplicity. The hard cases come up; we usually handle them in the write-up rather than in the data structure. The 90% rule is a working hypothesis: the coding scheme should serve the typical claim, and the analyst should not fail to remark on the unusual ones.

As a closing shot, we could say "Beyond minimalist causal coding, perhaps extended by opposites coding and despite coding, there is no hidden vocabulary (packages, mechanisms, necessity/sufficiency, blockers/enablers, causal strength and polarity) which can be reliably applied to causally coding an interestingly wide range range of texts at scale, no universal language for mid-range theory". But that would perhaps be a claim too far.

## References


Ackermann, Jones, Sweeney, & Eden (1996). *Decision Explorer: User Guide*. Vol. version, *3*.

Ackermann, Eden, & Cropper (2004). *Getting Started with Cognitive Mapping*.

Aston (2019). *Contribution Rubrics*.

Axelrod (1976). *Structure of Decision: The Cognitive Maps of Political Elites*. Princeton university press.

Axelrod (1976). *The Analysis of Cognitive Maps*. In *Structure of Decision : The Cognitive Maps of Political Elites*.

Braun, & Clarke (2021). *Thematic Analysis : A Practical Guide*. SAGE Publications Ltd.

Britt, Powell, & Caldas Cabral (2025). *Strengthening Outcome Harvesting with AI-Assisted Causal Mapping*.

Charmaz (2014). *Constructing Grounded Theory*. SAGE.

Copestake, Morsink, & Remnant (2019). *Attributing Development Impact: The Qualitative Impact Protocol Case Book*. Practical Action Publishing.

Eden, Ackermann, & Cropper (1992). *The Analysis of Cause Maps*. Journal of Management Studies, *29*, 309--324.

Gl{\"a (2013). *Life With and Without Coding: Two Methods for Early-Stage Data Analysis in Qualitative Research Aiming at Causal Explanations*. Forum Qualitative Sozialforschung / Forum: Qualitative Social Research, *Vol 14*, No 2 (2013). https://doi.org/10.17169/FQS-14.2.1886.

Hodgkinson, Maule, & Bown (2004). *Causal Cognitive Mapping in the Organizational Strategy Field: A Comparison of Alternative Elicitation Procedures*. Organizational Research Methods, *7*, 3--26.

Laukkanen (1994). *Comparative Cause Mapping of Organizational Cognitions*. Organization Science, *5*, 322--343.

Laukkanen (2012). *Comparative Causal Mapping and CMAP3 Software in Qualitative Studies*. Forum Qualitative Sozialforschung / Forum: Qualitative Social Research, *13*. https://doi.org/10.17169/fqs-13.2.1846.

Laukkanen, & Wang (2016). *Comparative Causal Mapping: The CMAP3 Method*. Routledge.

Lemire, Kwako, Nielsen, Christie, Donaldson, & Leeuw (2020). *What Is This Thing Called a Mechanism? Findings from a Review of Realist Evaluations*. Causal Mechanism in Evaluation, 73--86. https://doi.org/10.1002/ev.20428.

Lynn (2025). *HU Seafood Retrospective*.

Markiczy, & Goldberg (1995). *A Method for Eliciting and Comparing Causal Maps*. Journal of management, *21*, 305--333.

Maule, Hodgkinson, & Bown (2003). *Cognitive Mapping of Causal Reasoning in Strategic Decision Making*. In *Thinking: Psychological Perspectives on Reasoning, Judgment and Decision Making*. https://doi.org/10.1002/047001332X.ch13.

Mayring (2000). *Qualitative Content Analysis*. Forum Qualitative Sozialforschung / Forum: Qualitative Social Research, *Vol 1*, No 2 (2000): Qualitative Methods in Various Disciplines I: Psychology. https://doi.org/10.17169/FQS-1.2.1089.

Miles, Huberman, & Salda{\~n (2014). *Qualitative Data Analysis: A Methods Sourcebook*. SAGE Publications, Inc.

Pawson, & Tilley (1997). *Realistic Evaluation*. Sage Publications Limited.

Powell, & Larquemin (2023). *Does Our Theory Match Your Theory? Theories of Change and Causal Maps in Ghana*. In *Strategic Thinking, Design and the Theory of Change. A Framework for Designing Impactful and Transformational Social Interventions*. https://doi.org/10.4337/9781803927718.00025.

Powell, Copestake, & Remnant (2024). *Causal Mapping for Evaluators*. Evaluation, *30*, 100--119. https://doi.org/10.1177/13563890231196601.

Powell, Caldas Cabral, & Mishan (2025). *A Workflow for Collecting and Understanding Stories at Scale, Supported by Artificial Intelligence*. Evaluation, *31*, 394--411. https://doi.org/10.1177/13563890251328640.

Ragin (2008). *Measurement Versus Calibration: A Set-Theoretic Approach*. The Oxford Handbook of Political Methodology, 1--31. https://doi.org/10.1093/oxfordhb/9780199286546.003.0008.

Remnant, Copestake, Powell, & Channon (2025). *Qualitative Causal Mapping in Evaluations*. In *Handbook of Health Services Evaluation: Theories, Methods and Innovative Practices*. https://doi.org/10.1007/978-3-031-87869-5_12.

Salda{\~n (2015). *The Coding Manual for Qualitative Researchers*. Sage.

{Barbrook-Johnson (2022). *Participatory Systems Mapping*. In *Systems Mapping: How to Build and Use Causal Models of Systems*. https://doi.org/10.1007/978-3-031-01919-7_5.